

Basic C Without Tears

Guía Práctica Formativa – Tipo B

Estructuras, Punteros, Memoria Dinámica y Operaciones de Bits

Autor: Stack

1. Responda a las siguientes preguntas sobre estructuras, uniones y representación de datos compuestos en memoria.

- (a) Considere las siguientes dos definiciones en una arquitectura de 64 bits con alineación natural ($\text{sizeof}(\text{char}) = 1$, $\text{sizeof}(\text{int}) = 4$, $\text{sizeof}(\text{double}) = 8$):

```
struct Version_A {
    char    tipo;
    double  valor;
    int     codigo;
};

struct Version_B {
    double  valor;
    int     codigo;
    char    tipo;
};
```

Sin modificar los campos, indique cuál de las dos versiones ocupa menos bytes en memoria y justifique calculando el `sizeof` de cada una. Detalle los bytes de *padding* insertados y su ubicación.

- (b) El siguiente programa intenta copiar el contenido de una estructura a otra, pero el programador usó asignación campo a campo. Determine qué imprime el código y explique por qué la asignación directa `b = a;` entre variables del mismo tipo `struct` es válida en C:

```
1 struct Punto { int x; int y; };
2
3 struct Punto a = {15, -8};
4 struct Punto b;
5 b.x = a.x;
6 b.y = a.y;
7 a.x = 0;
8
9 printf("b: (%d, %d)\n", b.x, b.y);
```

- (c) Escriba una función en C con el siguiente prototipo:

```
int contar_aprobados(struct Alumno lista[], int n);
```

donde `struct Alumno` contiene los campos `char nombre[50]` y `float promedio`. La función debe retornar la cantidad de alumnos cuyo promedio sea mayor o igual a 4.0. Considere los siguientes casos de prueba:

Entrada	Salida esperada
{("Ana", 5.2), ("Luis", 3.9), ("Mia", 4.0)}	2
{("Pedro", 3.5), ("Sol", 2.8)}	0

2. Los punteros en C permiten manipular directamente las direcciones de memoria donde residen las variables. Cada tipo de puntero avanza en memoria según el tamaño del tipo al que apunta.

- (a) Determine qué imprime el siguiente fragmento de código. Justifique cada línea de salida indicando el valor almacenado y la operación realizada:

```
1 int datos[5] = {100, 200, 300, 400, 500};
2 int *p = datos + 1;
3
4 printf("%d\n", *p);
5 printf("%d\n", *(p + 2));
6 printf("%d\n", p[-1]);
7 p += 3;
8 printf("%d\n", *p);
```

- (b) El siguiente programa pretende intercambiar los valores de dos variables mediante una función, pero tras la llamada `x` e `y` permanecen inalteradas:

```
1 void intercambiar(int a, int b) {
2     int temp = a;
3     a = b;
4     b = temp;
5 }
6
7 int main() {
8     int x = 7, y = 13;
9     intercambiar(x, y);
10    printf("x=%d, y=%d\n", x, y);
11    return 0;
12 }
```

Explique por qué falla el intercambio y reescriba la función corregida utilizando punteros para que modifique los valores originales de `x` e `y`.

- (c) Escriba una función en C con el siguiente prototipo:

```
void separar_pares_impares(const int *origen, int n,
                           int *pares, int *np, int *impares, int *ni);
```

La función recorre el arreglo `origen` de `n` elementos y copia los números pares en el arreglo `pares` (almacenando la cantidad en `*np`) y los impares en el arreglo `impares` (almacenando la cantidad en `*ni`).

Entrada (origen)	Salida esperada
{3, 8, 15, 4, 7, 2}	pares: 8 4 2 (3), impares: 3 15 7 (3)
{10, 20, 30}	pares: 10 20 30 (3), impares: (0)

3. La gestión explícita de memoria dinámica en C se realiza mediante las funciones `malloc`, `realloc` y `free` de la biblioteca `<stdlib.h>`.

- (a) El siguiente código compila correctamente, pero contiene dos errores graves de gestión de memoria. Identifíquelos y explique sus consecuencias:

```
1  int *crear_secuencia(int n) {
2      int resultado[n];
3      for (int i = 0; i < n; i++) {
4          resultado[i] = (i + 1) * 10;
5      }
6      return resultado;
7  }
8
9  int main() {
10     int *sec = crear_secuencia(5);
11     for (int i = 0; i < 5; i++) {
12         printf("%d ", sec[i]);
13     }
14     free(sec);
15     return 0;
16 }
```

- (b) Reescriba la función `crear_secuencia` de forma correcta, reservando memoria en el Heap con `malloc` y verificando que la asignación no falle. Indique quién es responsable de liberar la memoria y dónde debe hacerse.
- (c) Escriba una función en C con el siguiente prototipo:

```
int *leer_hasta_cero(int *cantidad);
```

La función lee enteros desde la entrada estándar hasta que el usuario ingrese el valor 0 (centinela). Debe almacenar los valores leídos en un bloque dinámico que comienza con capacidad para 2 elementos y se duplica con `realloc` cada vez que se llena. Al finalizar, almacena en `*cantidad` el total de elementos leídos y retorna el puntero al bloque.

4. Las operaciones a nivel de bits (&, |, ^, ~, «, ») permiten manipular individualmente los bits de un valor entero. Son fundamentales en programación de sistemas, protocolos de comunicación y dispositivos embebidos.

- (a) Determine qué imprime el siguiente fragmento de código. Muestre el cálculo binario completo de cada operación (use representación de 8 bits):

```
1 unsigned char a = 0xC5; // 1100 0101
2 unsigned char b = 0x3A; // 0011 1010
3
4 printf("AND: 0x%02X\n", a & b);
5 printf("OR: 0x%02X\n", a | b);
6 printf("XOR: 0x%02X\n", a ^ b);
```

- (b) En un sistema de control de acceso, los permisos de un usuario se almacenan en una variable `unsigned char permisos` donde cada bit representa un permiso distinto:

■ Bit 0: Lectura Bit 1: Escritura Bit 2: Ejecución Bit 3: Administración

Un usuario tiene `permisos = 0x05` (binario: 0000 0101). Escriba las expresiones en C para:

1. Verificar si tiene permiso de Escritura (bit 1).
2. Otorgarle permiso de Administración (bit 3) sin alterar los demás.
3. Revocarle permiso de Lectura (bit 0) sin alterar los demás.
4. Conmutar (toggle) el permiso de Ejecución (bit 2).